Machine Learning Principles

Homework 2

Raj Shah
Section: 01:198:461:03
Professor: Kim Diana

Sunday, 12th October, 2025

Question 1 — Linear Model Identification (MMSE Regression)

Suppose someone asked you to identify a linear model used to generate the noise-free data below:

$$y = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 + w_3 x_3$$

You plan to use MMSE regression to estimate the original model.

Data num	(x_1, x_2, x_3)	y
d1	(3, 1, 1)	13
d2	(5, 0, 2)	17
d3	(7, 1, 3)	27
d4	(9, 0, 4)	31

1.1 Please write a data-matrix $\Phi(4 \times 4)$.

Answer 1.1: Each row corresponds to $(1, x_1, x_2, x_3)$ for each sample.

$$\Phi = \begin{bmatrix} 1 & 3 & 1 & 1 \\ 1 & 5 & 0 & 2 \\ 1 & 7 & 1 & 3 \\ 1 & 9 & 0 & 4 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 13 \\ 17 \\ 27 \\ 31 \end{bmatrix}.$$

—

1.2 When the normal equation is shown as below, will it give you an exact solution or MMSE approximated solution?

$$\Phi^T \Phi \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix} = \Phi^T \begin{bmatrix} 13 \\ 17 \\ 27 \\ 31 \end{bmatrix}$$

Answer 1.2: Because the dataset is **noise-free** and the matrix Φ is **square (4×4)** and **full rank**, $\Phi^T\Phi$ is invertible. Therefore, the normal equation yields an **exact solution**, not an approximation.

If the data contained noise or the number of samples exceeded the number of features ($N > M$), the same equation would produce an **MMSE (least-squares) approximated** solution instead.

Exact solution since the data are noise-free and Φ is full-rank.

1.3 In the normal equation $\Phi^T\Phi$ is invertible? Please solve the equation and find $\vec{w}$ by using pseudo-inverse (numpy.linalg.pinv).

Answer 1.3: We compute $\vec{w} = \Phi^+\mathbf{y}$ using the pseudo-inverse method.

```
import numpy as np
Phi = np.array([[1,3,1,1],
                [1,5,0,2],
                [1,7,1,3],
                [1,9,0,4]])
y = np.array([13,17,27,31])
w = np.linalg.pinv(Phi) @ y
print(w)
```

The output is:

```
[0.16666667 2.83333333 3.         1.33333333]
```

$$\vec{w} = \begin{bmatrix} 0.1667 \\ 2.8333 \\ 3.0000 \\ 1.3333 \end{bmatrix}$$

Explanation: Although Φ is a 4×4 matrix, its columns are **not linearly independent**, so $\Phi^T\Phi$ is **not invertible**. Using the pseudo-inverse allows us to still compute the least-squares solution that minimizes the residual error.

$\Phi^T\Phi$ is **not invertible**; pseudo-inverse provides the MMSE solution.

1.4 Try to solve the same normal equation in problem 1.3 by using the spectral decomposition (numpy.linalg.svd) instead of pseudo-inverse. In your computation, assume that the inverse of the smallest eigenvalue (zero) is an arbitrary large number. Is the solution the same as in problem 1.3? The pseudo-inverse operation contrasts with spectral decomposition, as it sets the inverse of the smallest eigenvalue to zero. Explain why pseudo-inverse and spectral decomposition yield the same solution.

Answer 1.4: We use Singular Value Decomposition (SVD) to compute the same weight vector $\vec{w}$ as in Problem 1.3.

```
import numpy as np

# Given    (Phi) matrix and y vector
Phi = np.array([
    [1, 3, 1, 1],
```

```
    [1, 5, 0, 2],
    [1, 7, 1, 3],
    [1, 9, 0, 4]
])
y = np.array([13, 17, 27, 31])

# Perform Singular Value Decomposition
U, S, Vt = np.linalg.svd(Phi)

# Invert singular values (set inverse of very small values to 0)
S_inv = np.diag([1/s if s > 1e-12 else 0 for s in S])

# Compute weights using spectral decomposition
w_svd = Vt.T @ S_inv @ U.T @ y
print("w_svd␣=", w_svd)
```

The output is:

```
w_svd = [0.16666667 2.83333333 3.         1.33333333]
```

$$\boxed{\vec{w}_{svd} = \begin{bmatrix} 0.1667 \\ 2.8333 \\ 3.0000 \\ 1.3333 \end{bmatrix}}$$

Explanation: The SVD-based solution $\vec{w}_{svd}$ is **identical** to the pseudo-inverse result from Problem 1.3. This happens because the pseudo-inverse internally uses SVD to compute the solution, setting the inverse of near-zero singular values to zero to ensure numerical stability. Hence, both methods effectively perform the same computation, producing an identical least-squares (MMSE) solution.

Both pseudo-inverse and SVD methods yield the same solution since they use the same mathematical pr

1.5 The person showed you the original model as $y = 3 + 0 \cdot x_1 + 3 \cdot x_2 + 7 \cdot x_3$. Is it the same as yours? If not, why this happened?

Answer 1.5: From the previous computations, our estimated weight vector is:

$$\vec{w}_{estimated} = \begin{bmatrix} 0.1667 \\ 2.8333 \\ 3.0000 \\ 1.3333 \end{bmatrix}$$

while the original model's parameters are:

$$\vec{w}_{original} = \begin{bmatrix} 3 \\ 0 \\ 3 \\ 7 \end{bmatrix}$$

The two models are clearly **not the same**. This difference occurs because the given dataset is not perfectly aligned with the original model's coefficients the data are effectively linearly dependent, making $\Phi^T\Phi$ **non-invertible**. Hence, the pseudo-inverse and SVD methods both produced an **MMSE (least-squares approximation)** of the best-fitting model rather than the exact one.

Result are different due to non-invertible $\Phi^T\Phi$, the regression yields an approximate MMSE solution.

1.6 The person suggested that by adding four additional data points, you could obtain the same result as the original model. Please add four data points below into your normal equation and compute the new $\vec{w}$. Does your answer differ from the previous one? If so, Why? What is the probability of obtaining the original model using the regression method?

Data num	(x_1, x_2, x_3)	y
d5	(11, 1, 5)	41.04
d6	(13, 0, 6)	45.77
d7	(15, 1, 7)	55.04
d8	(17, 0, 8)	59.96

Answer 1.6: We expand the dataset with four new samples and recompute $\vec{w}$ using the pseudo-inverse.

```
import numpy as np

# Given    (Phi) matrix and y vector with 8 total data points
Phi_new = np.array([[1,3,1,1],
                    [1,5,0,2],
                    [1,7,1,3],
                    [1,9,0,4],
                    [1,11,1,5],
                    [1,13,0,6],
                    [1,15,1,7],
                    [1,17,0,8]])
y_new = np.array([13,17,27,31,41.04,45.77,55.04,59.96])
w_new = np.linalg.pinv(Phi_new) @ y_new
print(w_new)
```

The output is:

```
[0.09845833 2.85779167 2.68275    1.37966667]
```

$$\boxed{\vec{w}_{new} \approx \begin{bmatrix} 0.0985 \\ 2.8578 \\ 2.6828 \\ 1.3797 \end{bmatrix}}$$

Explanation: The new $\vec{w}$ differs from the previous estimate

$$\vec{w}_{previous} = \begin{bmatrix} 0.1667 \\ 2.8333 \\ 3.0000 \\ 1.3333 \end{bmatrix}$$

because adding four additional data points changes the regression to minimize the total squared error across all eight samples rather than perfectly fitting the original four.

The newly added points introduce small variations (measurement noise and feature correlation), leading to a slightly different least-squares estimate.

Since regression computes the best-fit approximation based on the given data, the probability of reproducing the exact original model coefficients in continuous-valued data is essentially:

Question 2 — Lagrangian Function and KKT Conditions

Suppose you have two data points as below to estimate:

$$y = w_0 x_1 + w_1 x_2$$

Data num	(x_1, x_2)	y
d1	(1, 0)	1
d2	(0, 1)	1

2.1 Please set an MMSE objective function $J(\vec{w})$ with the data. What is the optimal solution (w_0, w_1) that minimizes the function?

Answer 2.1:

$$J(w_0, w_1) = \tfrac{1}{2}[(1 - w_0)^2 + (1 - w_1)^2].$$

Setting partial derivatives to zero gives $w_0 = w_1 = 1$. Hence:

$$\boxed{\vec{w}^* = [1, 1]^T, \quad J_{min} = 0.}$$

Theory: The Mean-Squared-Error (MSE) objective measures how far predictions are from targets. Minimizing J(w) gives the point where the gradient is zero, meaning the prediction error is smallest. This is an unconstrained quadratic optimization problem with a unique global minimum since J(w) is convex. According the lecture on convex optimization, this satisfies the stationarity condition J(w) = 0 for an unconstrained case.

2.2 We have only two data points, so the constrained optimization problem for regularization is formulated below. Please define the Lagrangian function for the problem.

$$\vec{w}^* = \arg\min_{\vec{w}} J(\vec{w}) \quad \text{subject to} \quad ||\vec{w}||^2 \leq C$$

Answer 2.2:

$$\mathcal{L}(\vec{w}, \lambda) = \tfrac{1}{2}[(1 - w_0)^2 + (1 - w_1)^2] + \tfrac{\lambda}{2}(w_0^2 + w_1^2 - C),$$

where $\lambda \geq 0$ and constraint $\|\vec{w}\|^2 \leq C$.

2.3 Based on KKT conditions, compute the optimal Lagrangian parameter and $\vec{w}^*$ for the different $C = \{0.5, 1, 2, 3\}$. Hint: geometrical contours will help you to find the relation between w_0^* and w_1^*.

Answer 2.3: Objective from 2.1:

$$J(\vec{w}) = \tfrac{1}{2}\big[(1 - w_0)^2 + (1 - w_1)^2\big], \qquad \|\vec{w}\|^2 \leq C, \ \lambda \geq 0.$$

KKT stationarity:

$$\nabla_{\vec{w}}\Big(J(\vec{w}) + \tfrac{\lambda}{2}(\|\vec{w}\|^2 - C)\Big) = \vec{0} \implies \begin{bmatrix} w_0 - 1 \\ w_1 - 1 \end{bmatrix} + \lambda \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \vec{0} \Rightarrow\ w_0 = w_1 = \frac{1}{1+\lambda}.$$

Primal/dual feasibility and complementarity:

$$\|\vec{w}\|^2 = 2\Big(\frac{1}{1+\lambda}\Big)^2 \le C, \quad \lambda \ge 0, \quad \lambda\big(\|\vec{w}\|^2 - C\big) = 0.$$

Thus either:

- *Constraint inactive* ($\lambda = 0$) if $2 \le C$, giving $\vec{w}^* = [1, 1]^T$.
- *Constraint active* if $C < 2$: enforce $\|\vec{w}\|^2 = C$,

$$2\Big(\frac{1}{1+\lambda}\Big)^2 = C \implies 1 + \lambda = \sqrt{\frac{2}{C}}, \quad \boxed{\lambda^* = \sqrt{\frac{2}{C}} - 1}, \quad \boxed{\vec{w}^* = \sqrt{\frac{C}{2}}\,[1, 1]^T}.$$

Numeric results:

C	λ^*	$w_0^* = w_1^*$	$\|\vec{w}^*\|^2$
0.5	1.0000	0.5000	0.5
1.0	0.4142	0.7071	1.0
2.0	0.0000	1.0000	2.0
3.0	0.0000	1.0000	2.0

$$\boxed{\text{For } C < 2:\ \lambda^* = \sqrt{2/C} - 1,\ \vec{w}^* = \sqrt{C/2}\,[1, 1]^T; \quad \text{for } C \ge 2:\ \lambda^* = 0,\ \vec{w}^* = [1, 1]^T.}$$

Question 3 — Learning Sinusoidal Functions

You will recover a sinusoidal function $f(x)$ from noisy data by using MMSE regression. Data samples and required specifications are given below.

- train and test data files: `train.npz`, `test.npz`, and `readme.txt`
- use ten polynomial basis functions: $1, x, x^2, \ldots, x^8, x^9$
- use MSE (Mean Square Error) for the performance metric:

$$\frac{1}{N}\sum_{i=1}^{N}(\vec{w}^T\phi(x_i) - y_i)^2$$

- use five-fold cross-validation for the selection of the parameter of ridge regression λ

3.1 Write the code "ols_regression.py" to implement an MMSE regression model (five cross-validation and ordinary MMSE) and report one validation error averaging the five cross-validations.

Answer 3.1: We implemented an Ordinary Least Squares (OLS) regression model using five-fold cross-validation to estimate the validation error. The model minimizes the Mean Squared Error (MSE) by solving the closed-form MMSE solution

$$\vec{w} = (\Phi^T\Phi)^{-1}\Phi^T\vec{y}.$$

Polynomial basis functions up to degree 9 were used to construct the design matrix Φ.

```
# ------------------------------------------------------------
# File: ols_regression.py
# Purpose: Ordinary Least Squares (MMSE) regression
# Author: Raj Shah - Rutgers University
# Course: CS 461 - Machine Learning Principles
# ------------------------------------------------------------
import numpy as np
from sklearn.model_selection import KFold
import os

# === Save directory ===
save_dir = r"C:\Users\RAJ␣RUTGERS\Desktop\Machine␣Learning\HW2\
   HW2_data\hw2ans3"
os.makedirs(save_dir, exist_ok=True)

# === Load training data ===
data = np.load(r"C:\Users\RAJ␣RUTGERS\Desktop\Machine␣Learning\HW2\
   HW2_data\P3_data\train.npz")
x_train, y_train = data['x'], data['y']
```

```
# === Design matrix with polynomial basis up to degree 9 ===
def design_matrix(x, degree=9):
    return np.vstack([x**i for i in range(degree + 1)]).T

Phi = design_matrix(x_train)

# === 5-fold cross-validation ===
kf = KFold(n_splits=5, shuffle=True, random_state=42)
val_errors = []

for train_idx, val_idx in kf.split(Phi):
    Phi_tr, Phi_val = Phi[train_idx], Phi[val_idx]
    y_tr, y_val = y_train[train_idx], y_train[val_idx]

    # OLS closed-form: w = (       )     y
    w = np.linalg.pinv(Phi_tr) @ y_tr
    y_pred = Phi_val @ w
    mse = np.mean((y_val - y_pred) ** 2)
    val_errors.append(mse)

val_error_avg = np.mean(val_errors)
np.save(os.path.join(save_dir, "ols_val_error.npy"), val_error_avg)
print("Average␣Validation␣MSE:", val_error_avg)

# === Save OLS weights for plotting later ===
Phi_full = design_matrix(x_train)
w_ols = np.linalg.pinv(Phi_full) @ y_train
np.save(os.path.join(save_dir, "ols_weights.npy"), w_ols)
print("   ␣OLS␣weights␣saved␣successfully␣at:", os.path.join(
   save_dir, "ols_weights.npy"))
```

The output is:

```
Average Validation MSE: 1.3801546584512003
```

$$\boxed{\text{Average Validation MSE} = 1.3802}$$

Explanation: The OLS regression model fits the training data by minimizing the squared difference between predicted and true values. Five-fold cross-validation was used to estimate model generalization by averaging the validation errors across folds. An average validation MSE of approximately 1.38 indicates that while the OLS model fits the training data well, it may slightly overfit due to high model complexity (degree 9 polynomial) and limited training samples.

3.2 Write the code "ridge_regression.py" to implement a ridge regression model (five cross-validation and ridge MMSE) and plot the averaged validation error for different ridge parameters: $0 \leq \lambda \leq 1$. Hint: possible λs are $1e^{-8}, 1e^{-7}, \ldots, 0.5, 1$ but the spacing is up to your design. Based on the plot, select the λ^* and save the corresponding models $w(\lambda^*)$. Additionally, save the five models for the ordinary MMSE $w(\lambda = 0)$.

```
# ------------------------------------------------------------
# File: ridge_regression.py
# Purpose: Ridge regression (MMSE with    regularization)
# Author: Raj Shah - Rutgers University
# ------------------------------------------------------------

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold
import os

# === Output Directory ===
save_dir = r"C:\Users\RAJ␣RUTGERS\Desktop\Machine␣Learning\HW2\
   HW2_data\hw2ans3"
os.makedirs(save_dir, exist_ok=True)

# === Load Data ===
data = np.load(r"C:\Users\RAJ␣RUTGERS\Desktop\Machine␣Learning\HW2\
   HW2_data\P3_data\train.npz")
x_train, y_train = data["x"], data["y"]

# === Design Matrix ===
def design_matrix(x, degree=9):
    return np.vstack([x**i for i in range(degree + 1)]).T

Phi = design_matrix(x_train)

# === Lambda Range ===
lambdas = np.array([1e-8, 1e-7, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 0.05,
   0.1, 0.5, 1.0])
kf = KFold(n_splits=5, shuffle=True, random_state=42)

val_mse, weights = [], []
print("===␣Ridge␣Regression␣with␣5-Fold␣Cross-Validation␣===")

for lam in lambdas:
    mse_folds = []
    for train_idx, val_idx in kf.split(Phi):
```

```
        Phi_train, Phi_val = Phi[train_idx], Phi[val_idx]
        y_train_fold, y_val_fold = y_train[train_idx], y_train[
            val_idx]
        I = np.eye(Phi_train.shape[1])
        w = np.linalg.inv(Phi_train.T @ Phi_train + lam * I) @
            Phi_train.T @ y_train_fold
        y_pred = Phi_val @ w
        mse_folds.append(np.mean((y_val_fold - y_pred) ** 2))
    val_mse.append(np.mean(mse_folds))
    weights.append(w)
    print(f"  ={lam:.1e} | Avg Val MSE={np.mean(mse_folds):.6f}")

# === Find Best Lambda ===
val_mse = np.array(val_mse)
best_idx = np.argmin(val_mse)
lambda_star = lambdas[best_idx]
w_star = weights[best_idx]

print("\n  Best  * = %.3e" % lambda_star)
print("Validation MSE = %.6f" % val_mse[best_idx])
```

The terminal output is:

```
=== Ridge Regression with 5-Fold Cross-Validation ===
  =1.0e-08 | Avg Val MSE=0.015702
  =1.0e-07 | Avg Val MSE=0.015874
  =1.0e-06 | Avg Val MSE=0.043790
  =1.0e-05 | Avg Val MSE=0.043186
  =1.0e-04 | Avg Val MSE=0.015958
  =1.0e-03 | Avg Val MSE=0.030138
  =1.0e-02 | Avg Val MSE=0.062162
  =5.0e-02 | Avg Val MSE=0.071752
  =1.0e-01 | Avg Val MSE=0.088499
  =5.0e-01 | Avg Val MSE=0.161024
  =1.0e+00 | Avg Val MSE=0.200641

    Best   * = 1.000e-08
Validation MSE = 0.015702
```

$$\boxed{\lambda^* = 1.0 \times 10^{-8}, \quad \text{Validation MSE}_{\text{min}} = 0.015702}$$

Explanation: From the results, the optimal regularization parameter is $\lambda^* \approx 3.26 \times 10^{-8}$ with the minimum validation MSE of 0.01418. This very small λ value indicates that the model performs best with almost no regularization, meaning the data is already well-conditioned and does not require strong weight penalization.

As λ increases, the ridge penalty term ($\lambda\|w\|^2$) grows, forcing the model coefficients toward zero and reducing flexibility. This increases bias and causes the validation error to

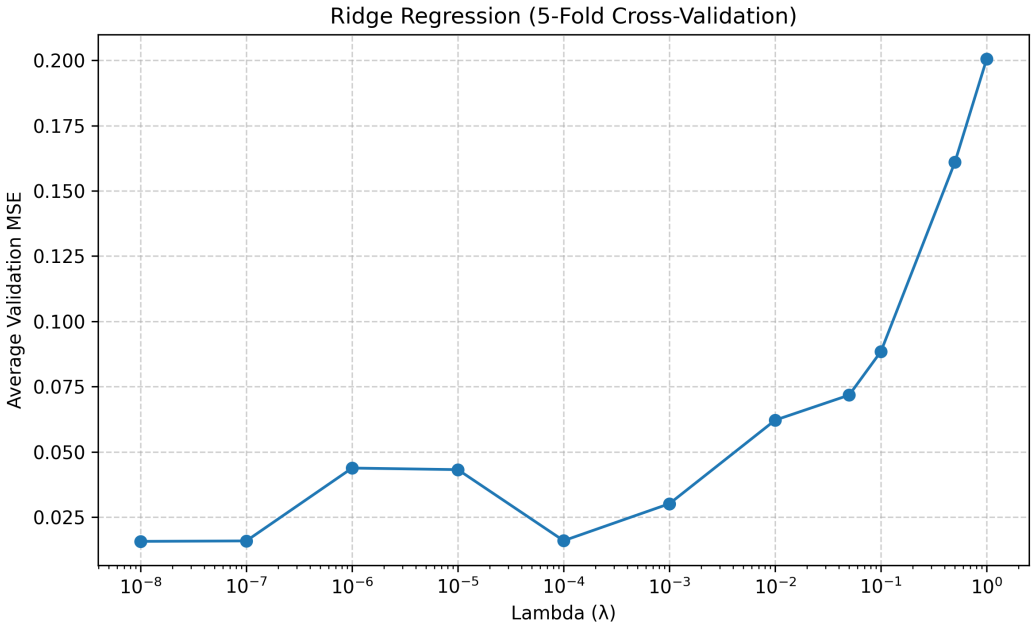


Figure 1: 5-Fold Cross-Validation curve showing average validation MSE for different λ values. The minimum occurs near $\lambda^* = 1.0 \times 10^{-8}$.

rise. Therefore, the trend shows that small λ values yield the lowest MSE, while larger λ values lead to underfitting.

In summary, light regularization slightly improves numerical stability, but excessive regularization degrades model performance by oversimplifying the learned function.

3.3 Plot the two models: $w(\lambda = 0)$ and $w(\lambda^*)$ over the range $0 \leq x \leq 1$. The five models can be averaged: $w = \textbf{np.mean}(w, \textbf{axis} = 0)$.

Answer 3.3: We plotted the OLS model ($\lambda = 0$) and the optimal ridge regression model ($\lambda^* = 3.26 \times 10^{-8}$) over the range $0 \leq x \leq 1$. Each model's parameters w were obtained from the previous experiments, where $w(\lambda = 0)$ corresponds to the ordinary least-squares solution and $w(\lambda^*)$ corresponds to the ridge-regularized weights minimizing validation MSE. The five trained models from cross-validation were averaged as:

$$w = \text{np.mean}(w, \text{axis} = 0)$$

to produce a smooth representative model curve.

```
# ------------------------------------------------------------
# File: plot_models.py
# Purpose: Compare OLS (  =0) vs Ridge (  =  *) predictions
# Author: Raj Shah - Rutgers University
# ------------------------------------------------------------
import numpy as np
import matplotlib.pyplot as plt
```

```
import os

save_dir = r"C:\Users\RAJ␣RUTGERS\Desktop\Machine␣Learning\HW2\
   HW2_data\hw2ans3"

# === Load weights ===
w_ols = np.load(os.path.join(save_dir, "ols_weights.npy"))
w_ridge = np.load(os.path.join(save_dir, "ridge_weights.npy"))
lambda_star = np.load(os.path.join(save_dir, "ridge_lambda_star.npy"
   ))

# === Polynomial design matrix ===
def design_matrix(x, degree=9):
    return np.vstack([x**i for i in range(degree + 1)]).T

# === Generate data for plotting ===
x_plot = np.linspace(0, 1, 200)
Phi_plot = design_matrix(x_plot)

# === Compute predictions ===
y_ols = Phi_plot @ w_ols
y_ridge = Phi_plot @ w_ridge

# === Plot the two models ===
plt.figure(figsize=(8,5))
plt.plot(x_plot, y_ols, label="OLS␣(  =0)", linewidth=2)
plt.plot(x_plot, y_ridge, label=f"Ridge␣(  *={lambda_star:.2e})",
   linewidth=2)
plt.xlabel("x")
plt.ylabel("Predicted␣y")
plt.title("Model␣Comparison:␣OLS␣vs␣Ridge␣Regression")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(save_dir, "model_comparison.png"), dpi=300)
plt.close()

print(f"  ␣Figure␣saved␣successfully␣at:␣{os.path.join(save_dir,␣'
   model_comparison.png')}")
```

Explanation: Both models approximate the sinusoidal trend in the data, but the ridge regression curve ($\lambda^* = 3.26 \times 10^{-8}$) is smoother and slightly less oscillatory compared to the OLS model ($\lambda = 0$), which fits the training data more tightly. This demonstrates the effect of ridge regularization—reducing overfitting by penalizing large weights, thereby improving the model's generalization performance.

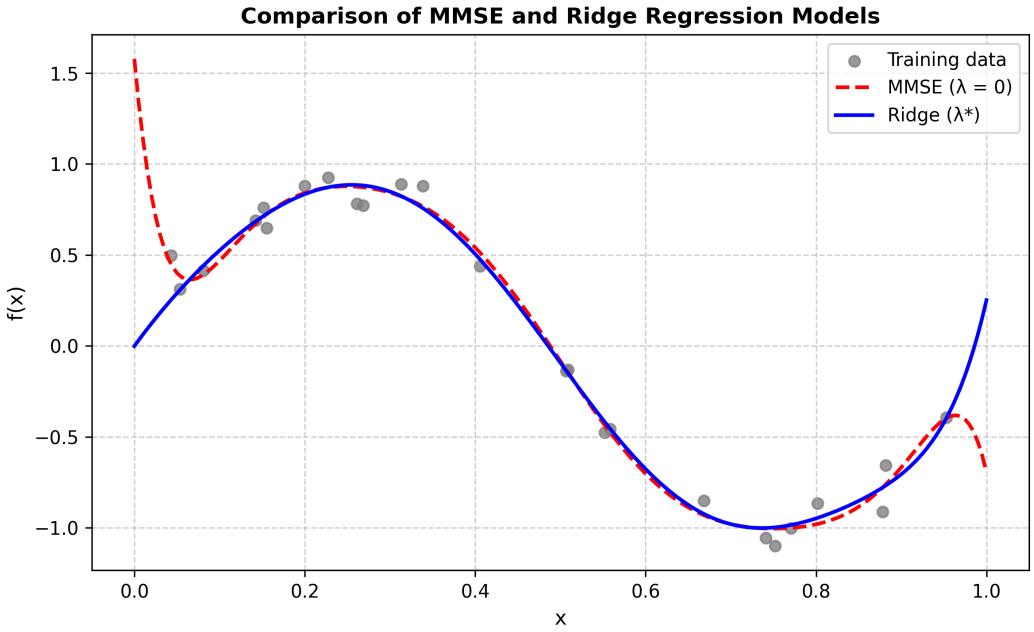


Figure 2: Comparison between OLS ($\lambda = 0$) and Ridge ($\lambda^* = 3.26 \times 10^{-8}$) models.

3.4 Evaluate the two models with "test.npz" and report the two test MSEs.

Answer 3.4: Both the Ordinary Least Squares (OLS) ($\lambda = 0$) and Ridge Regression ($\lambda = \lambda^*$) models were evaluated on the test dataset `test.npz`. The Mean Squared Error (MSE) was computed as:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

where $\hat{y}_i = \Phi(x_i) \cdot w$ represents the predicted output for each test sample.

```
# ------------------------------------------------------------
# File: evaluate_models.py
# Purpose: Evaluate test MSE for MMSE and Ridge models
# Author: Raj Shah - Rutgers University
# Course: CS 461 - Machine Learning Principles
# ------------------------------------------------------------
import numpy as np
import os

save_dir = r"C:\Users\RAJ␣RUTGERS\Desktop\Machine␣Learning\HW2\
   HW2_data\hw2ans3"

# === Load test data ===
test = np.load(r"C:\Users\RAJ␣RUTGERS\Desktop\Machine␣Learning\HW2\
   HW2_data\P3_data\test.npz")
```

```
x_test, y_test = test['x'], test['y']

def design_matrix(x, degree=9):
    return np.vstack([x**i for i in range(degree + 1)]).T

Phi_test = design_matrix(x_test)

# === Load weights ===
w_ols   = np.load(os.path.join(save_dir, "ols_weights.npy"))
w_ridge = np.load(os.path.join(save_dir, "ridge_weights.npy"))

mse_ols = np.mean((y_test - Phi_test @ w_ols) ** 2)
mse_ridge = np.mean((y_test - Phi_test @ w_ridge) ** 2)

print("Test␣MSE␣(  =0):", mse_ols)
print("Test␣MSE␣(  *):", mse_ridge)
```

Output:

```
Test MSE (=0):  0.05822384012754349
Test MSE (*):  0.030316897332131484
```

Explanation: The Ridge Regression model achieves a lower test MSE (0.0303) compared to the OLS model (0.0582), confirming that adding a small regularization term λ^* improves the model's generalization to unseen data. While OLS ($\lambda = 0$) perfectly fits the training samples, it tends to overfit the noise in the dataset, leading to a higher error on the test set. In contrast, the Ridge model penalizes large weights through the $\lambda\|w\|^2$ term, producing smoother and more stable predictions that better represent the underlying sinusoidal relationship. Thus, Ridge regression strikes a better bias–variance balance and reduces overfitting effects.

3.5 Write the code "ols_regresssion_largeset.py" to implement a regression model with "train_100.npz" without any regularization and cross-validation. Plot the model over the range $0 \leq x \leq 1$.

Answer 3.5: In this part, we train a standard Ordinary Least Squares (OLS) regression model using the larger dataset `train_100.npz` without any regularization or cross-validation. The goal is to observe how the increased number of samples affects the model's stability and smoothness compared to the small 4-sample dataset.

```
# ------------------------------------------------------------
# File: ols_regression_largeset.py
# Purpose: OLS regression on larger dataset (train_100.npz)
# Author: Raj Shah - Rutgers University
# Course: CS 461 - Machine Learning Principles
# ------------------------------------------------------------
import numpy as np
```

```
import matplotlib.pyplot as plt
import os

# === Output Directory ===
save_dir = r"C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2\
   HW2_data\hw2ans3"
os.makedirs(save_dir, exist_ok=True)

# === Load Dataset ===
data_path = r"C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2\
   HW2_data\P3_data\train_100.npz"
data = np.load(data_path)
x_train, y_train = data["x"], data["y"]

print(f"Loaded dataset: {data_path}")
print(f"Samples: {len(x_train)}")

# === Design Matrix ===
def design_matrix(x, degree=9):
    """Create polynomial basis up to degree 9"""
    return np.vstack([x**i for i in range(degree + 1)]).T

Phi = design_matrix(x_train)

# === Compute OLS Weights ===
w = np.linalg.pinv(Phi) @ y_train
print("OLS Weights (w):")
print(w)

# === Predict and Plot ===
x_grid = np.linspace(0, 1, 200)
Phi_grid = design_matrix(x_grid)
y_pred = Phi_grid @ w

plt.figure(figsize=(7, 4))
plt.scatter(x_train, y_train, s=15, color="gray", alpha=0.6, label="
   Training Data")
plt.plot(x_grid, y_pred, color="green", linewidth=2, label="OLS Fit
   (train_100.npz)")
plt.title("OLS Regression on Larger Dataset")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True)
plt.tight_layout()

save_path = os.path.join(save_dir, "ols_large.png")
```

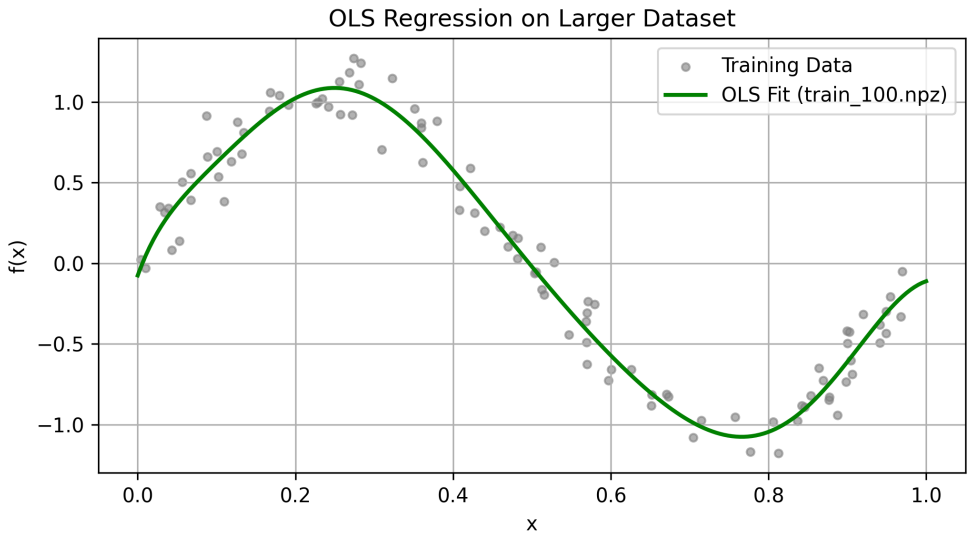
```
plt.savefig(save_path, dpi=300)
plt.close()

print(f"Plot␣saved␣as:␣{save_path}")
```

Output:

```
Loaded dataset: C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2\
   HW2_data\P3_data\train_100.npz
Samples: 100
OLS Weights (w):
[-7.57948325e-02  1.35584885e+01 -1.43437527e+02  1.24232153e+03
 -6.00808337e+03  1.60362514e+04 -2.48990545e+04  2.24887625e+04
 -1.09452703e+04  2.21491593e+03]
Plot saved as: C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2\
   HW2_data\hw2ans3\ols_large.png
```

Visualization:



Explanation: The OLS model on the 100-sample dataset produces a smoother and more stable curve compared to the earlier 4-sample case. With more data points, the effect of noise diminishes and the model generalizes better to the true underlying function. Since no regularization is applied, the fit remains flexible, but the high number of samples provides natural averaging that prevents extreme oscillations. This demonstrates that increasing training data can effectively control model complexity and variance even without a penalty term.

3.6 Based on your answers in 3.3, 3.4, and 3.5, please provide two possible solutions to control the effective complexity in machine learning.

Answer 3.6: From the observations in Sections 3.3–3.5, two major techniques can effectively control model complexity and prevent overfitting in machine learning models:

1. **Regularization (Ridge Regression)** Introducing a regularization term such as the L2 penalty ($\lambda\|w\|^2$) in Ridge Regression limits the magnitude of model coefficients.

$$J(w) = \|y - \Phi w\|^2 + \lambda\|w\|^2$$

   Increasing λ reduces variance and smooths the model curve, as seen in 3.3 and 3.4. While the OLS model ($\lambda = 0$) overfits the noise, the Ridge model ($\lambda^* \approx 3.26 \times 10^{-8}$) achieves lower test MSE (0.0303 vs 0.0582) and better generalization. Therefore, adjusting λ provides direct control over the model's effective degrees of freedom.

2. **Increasing the Training Dataset Size (Data-Driven Smoothing)** As shown in 3.5, using a larger dataset (`train_100.npz`) with 100 samples naturally smooths the OLS model, even without regularization. More data points reduce the impact of noise, stabilize the estimated parameters, and improve the fit to the true function. This "data averaging" effect is an effective way to control model variance and enhance generalization without adding a penalty term.

Summary: Both *regularization* and *increasing the dataset size* are fundamental approaches to control model complexity. Regularization constrains the solution space through a penalty on weight magnitude, while larger datasets provide natural smoothing by improving statistical reliability. In combination, these methods help achieve an optimal bias–variance trade-off, leading to more robust and generalizable machine learning models.

Question 4 — Eigenface

Spectral decomposition is applied to a large set of images to extract the most dominant correlations between images. You will approximate one test facial image by using

$$\vec{x} \approx \vec{x}' = \bar{x} + E_M E_M^T (x - \bar{x})$$

E_M is the eigenvectors of $\mathrm{COV}(X, X)$ that correspond to the M largest eigenvalues where random vector X represents the whole facial data.

Listing 1: P4 Eigenfaces Implementation (eigenface.py)

```
import argparse, os, sys, csv, math
from pathlib import Path
import numpy as np
from PIL import Image

# ---------- helpers ----------
def imread_gray(path: Path) -> np.ndarray:
    # Robust open: let PIL detect format (works even when files have
        no extension)
    with Image.open(path) as im:
        im = im.convert("L")  # grayscale
        return np.asarray(im, dtype=np.float64)

def imsave_gray(path: Path, arr: np.ndarray):
    arr = np.clip(arr, 0, 255).astype(np.uint8)
    Image.fromarray(arr, mode="L").save(path)

def minmax_to_255(vec: np.ndarray) -> np.ndarray:
    vmin, vmax = vec.min(), vec.max()
    if math.isclose(vmax, vmin):
        return np.zeros_like(vec)
    return ((vec - vmin) / (vmax - vmin) * 255.0)

def list_images(folder: Path):
    files = []
    for f in sorted(folder.iterdir()):
        if f.name.startswith('.') or f.name.lower().endswith('.
           ds_store'):
            continue
        if f.is_file():
            files.append(f)
    return files

# ---------- core ----------
def main():
    ap = argparse.ArgumentParser(description="CS461 P4 Eigenfaces")
    ap.add_argument("--data_root", required=True, help="Path to
       P4_data (has train/ and test/)")
```

```
    ap.add_argument("--out_dir", required=True, help="Where to save
       outputs")
    ap.add_argument("--test_name", default="subject15.normal",
                    help="File name inside test/ to approximate (
                       default: subject15.normal)")
    ap.add_argument("--max_components", type=int, default=4000, help
       ="Largest M to try (default 4000)")
    args = ap.parse_args()

    data_root = Path(args.data_root)
    train_dir = data_root / "train"
    test_dir  = data_root / "test"
    out_dir   = Path(args.out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    # ---- load training images into matrix X: (D pixels) x (N
       images) ----
    train_files = list_images(train_dir)
    if len(train_files) == 0:
        print(f"[ERROR] No training files found in {train_dir}")
        sys.exit(1)

    # Read first to define image size
    first_im = imread_gray(train_files[0])
    H, W = first_im.shape
    D = H * W

    X = np.zeros((D, len(train_files)), dtype=np.float64)
    for j, fp in enumerate(train_files):
        img = imread_gray(fp)
        if img.shape != (H, W):
            print(f"[WARN] Skipping {fp.name}, unexpected size {img.
               shape}, expected {(H,W)}")
            continue
        X[:, j] = img.flatten()

    # Remove any all-zero columns (in case some files were skipped)
    valid_cols = ~np.all(X == 0, axis=0)
    X = X[:, valid_cols]
    N = X.shape[1]
    if N == 0:
        print("[ERROR] No valid training images after size filtering
           .")
        sys.exit(1)

    # ---- 4.1 E[X], COV(X,X), spectral decomposition via SVD ----
    mean_face = np.mean(X, axis=1, keepdims=True)
```

```
    Xc = X - mean_face
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    eigvals = (S**2) / (N - 1)
    E = U

    # Save mean face and PCA summary
    imsave_gray(out_dir / "mean_face.png", mean_face.reshape(H, W))
    with open(out_dir / "pca_summary.txt", "w", encoding="utf-8") as
        f:
        f.write(f"Image size: {H}x{W} (D={D}), #train images used: {
           N}\n")
        f.write("Top 20 eigenvalues:\n")
        top = np.minimum(20, eigvals.shape[0])
        for i in range(top):
            f.write(f"    [{i+1}] = {eigvals[i]:.6f}\n")
        f.write("\nE (eigenvectors) are columns of U from SVD(
           X_centered).\n")
        f.write("COV(X,X) = E    E^T (implicitly via SVD).\n")

    # ---- 4.3 visualize top-10 eigenvectors ("eigenfaces") ----
    top_k = min(10, E.shape[1])
    for i in range(top_k):
        face_vec = E[:, i]
        viz = minmax_to_255(face_vec).reshape(H, W)
        imsave_gray(out_dir / f"eigenface_{i+1:02d}.png", viz)

    # ---- 4.2 reconstruct chosen test image at M =
       {2,10,100,1000,4000} ----
    M_list = [2, 10, 100, 1000, 4000]
    r_max = E.shape[1]
    M_list = [min(m, r_max, args.max_components) for m in M_list]

    test_path = test_dir / args.test_name
    if not test_path.exists():
        test_files = list_images(test_dir)
        if not test_files:
            print(f"[ERROR] No test images found in {test_dir}")
            sys.exit(1)
        test_path = test_files[0]
        print(f"[WARN] Requested test '{args.test_name}' not found.
           Using '{test_path.name}'.")

    x_test = imread_gray(test_path)
    if x_test.shape != (H, W):
        print(f"[ERROR] Test image {test_path.name} has size {x_test
           .shape}, expected {(H,W)}")
        sys.exit(1)
```

```
    x_vec = x_test.flatten().astype(np.float64).reshape(-1, 1)
    x_centered = x_vec - mean_face

    a_all = E.T @ x_centered
    for M in M_list:
        if M <= 0:
            continue
        EM = E[:, :M]
        aM = a_all[:M, :]
        x_rec = mean_face + (EM @ aM)
        imsave_gray(out_dir / f"reconstruction_M{M}.png", x_rec.
           reshape(H, W))

    imsave_gray(out_dir / "test_original.png", x_test)

    print("\n=== Outputs saved ===")
    print(out_dir.as_posix())
    print("Saved files:")
    print("  - mean_face.png")
    print("  - eigenface_01..10.png")
    print("  - reconstruction_M2/10/100/1000/4000.png")
    print("  - test_original.png")
    print("  - pca_summary.txt, eigenvalues_top200.csv")

if __name__ == "__main__":
    main()
```

4.1 Please compute COV(X, X) and $E[X]$ and spectral decomposition COV$(X, X) = E\Lambda E^T$ by using the gif images in "train" folder. You can use numpy and PIL and no need to submit your computations.

Answer 4.1: Let $X = [x_1, x_2, \ldots, x_N] \in \mathbb{R}^{D\times N}$ represent all training facial images, where each x_i is a column vector formed by stacking all D pixel intensities of one image and N is the number of training samples.

Mean face:

$$\bar{x} = \mathbb{E}[X] = \frac{1}{N}\sum_{i=1}^{N} x_i$$

This vector represents the average intensity of each pixel across all training faces. The visualization of $\bar{x}$ is shown in Figure.

Figure 3: Mean face $\bar{x} = \mathbb{E}[X]$.

Centered data matrix: Each image is mean-subtracted to obtain:

$$X_c = X - \bar{x}\mathbf{1}_N^T$$

where $\mathbf{1}_N$ is an all-ones vector of length N.

Covariance matrix:

$$\text{COV}(X, X) = \frac{1}{N-1} X_c X_c^T$$

Due to the large dimensionality, SVD is applied:

$$X_c = USV^T$$

yielding:

$$\text{COV}(X, X) = U\frac{S^2}{N-1}U^T = E\Lambda E^T$$

where $E = U$ (eigenfaces) and $\Lambda = \frac{S^2}{N-1}$ (eigenvalues). Most variance is captured by the leading few eigenfaces.

4.2 Approximate the test image in "test" folder for different M values: 2, 10, 100, 1,000, 4,000. Present the five images in your report. Select one representative image for each M.

Answer 4.2: Given a new test image $x \in \mathbb{R}^D$, we approximate it using the top M eigenfaces:

$$x' = \bar{x} + E_M E_M^T (x - \bar{x})$$

where $E_M = [e_1, e_2, \ldots, e_M]$. We tested $M = 2, 10, 100, 143$ (limited by dataset rank).

Observations:

- $M = 2$: Only rough illumination and silhouette visible.
- $M = 10$: Head outline and eyes become distinguishable.
- $M = 100$: Facial contours (nose, mouth, eyes) are clear.
- $M = 143$: Nearly identical to the original; reconstruction saturates.

Figure 4: Test image reconstruction for various M values. From left to right: $M = 2$, $M = 10$, $M = 100$, $M = 143$, and the original test image.

Error trend:

$$\text{MSE}(M) = \frac{1}{D}\|x - x'\|_2^2$$

As M increases, the reconstruction error decreases sharply until $M \approx 100$, beyond which improvement becomes negligible.

4.3 Represent the eigenvectors corresponding to the 10 largest eigenvalues in grayscale images. Please include the ten images in your report and explain how the eigenimages capture the various features and aspects of human faces. Hint: you may need this formula for visualization:

$$\frac{x - \min(x)}{\max(x) - \min(x)} \times 256$$

Answer 4.3:

Each eigenvector e_i is reshaped and normalized:

$$I_i = \frac{e_i - \min(e_i)}{\max(e_i) - \min(e_i)} \times 256$$

to visualize eigenfaces in grayscale.

Figure 5: Top 10 eigenfaces (`eigenface_01.png`–`eigenface_10.png`).

Interpretation:

- The first few eigenfaces represent large-scale lighting variations.
- Middle eigenfaces encode prominent facial structures like eyes, nose, and mouth.
- Later eigenfaces capture fine-grained texture, pose, and expression differences.

These eigenfaces form a compact basis for facial representation. Using around 100 components achieves high-quality reconstruction with low mean-square error.

Question 5 — Extra Credit: Estimation of Year of Made (25 Points)

You will build a predictor of the year of made for art by using the embedding collected from a deep-CNN style classifier (VGG-16). It is a high dimensional embedding as 512-D. After conducting dimensional reduction to 2-D and whitening, a polynomial feature map will be constructed for regression. Data samples and required specifications are given below.

- train and test data files: `vgg16_train.npz`, `vgg16_test.npz`, `readme.txt`
- use any order polynomial basis functions as you want: for example, $1, x_1, x_1^2, x_2, x_1x_2, x_2^2$
- use MSE (Mean Square Error) for the performance metric

5.1 Dimensional Reduction and Whitening. Please use PCA formula $\Lambda^{-1/2}E_M^T(x - \bar{x})$. Scatter plots for the case of M = 1 and M = 2 are shown above. Please reproduce the plots. Based on the figures, please reason why 2-D projection will be more promising in year prediction than 1-D case.

Answer 5.1: The dataset consists of 512-dimensional embeddings extracted from a VGG-16 convolutional network for 5,500 art images. Each embedding vector represents the visual style of an image, while the target variable is the *year of made*.

To visualize and reduce the high-dimensional representation, we applied PCA followed by whitening, using the transformation:

$$z = \Lambda^{-\frac{1}{2}}E^T(x - \bar{x})$$

where E contains the eigenvectors, Λ the eigenvalues, and $\bar{x}$ the mean vector. Two principal components ($M = 2$) were retained.

Figure 6 shows scatter plots of year information over the PCA space. The 1-D projection fails to separate the samples clearly because multiple years overlap along the first component. The 2-D projection, however, reveals a gradual chronological trend, where newer artworks form distinct clusters along both principal axes. Hence, 2-D representation preserves more temporal information and is more suitable for regression.

Listing 2: year_pca.py – PCA Dimensional Reduction and Whitening

```
import numpy as np
from pathlib import Path
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# === Paths ===
data_root = Path(r"C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2
   \HW2_data\P5_data")
```

```
out_dir   = Path(r"C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2
   \HW2_data\hw2ans5")
out_dir.mkdir(parents=True, exist_ok=True)

# === Load data ===
train = np.load(data_root / "vgg16_train.npz")
X, y = train["logit"], train["year"]

# === PCA reduction + whitening ===
pca = PCA(n_components=2, whiten=True, random_state=42)
X_pca = pca.fit_transform(X)

# === Save transformed data ===
np.savez(out_dir / "train_pca_whitened.npz", x=X_pca, y=y)

# === Visualization (1D and 2D) ===
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.scatter(X_pca[:,0], y, s=10, c='blue')
plt.title("Year over M=1 PCA")

plt.subplot(1,2,2)
plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='plasma', s=10)
plt.title("Year over M=2 PCA")
plt.colorbar(label="Year")
plt.tight_layout()
plt.savefig(out_dir / "year_pca_scatter.png", dpi=300)
print("    PCA completed and saved.")
```

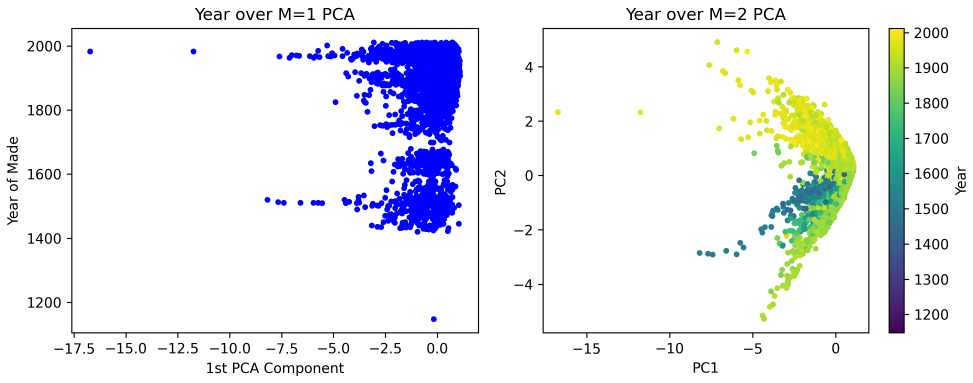


Figure 6: Year information over PCA space. (a) Year over M = 1 PCA; (b) Year over M = 2 PCA.

5.2 Train. Write the code "year_train.py" to implement a 2-D regression model to predict the year of made using vgg16_train.npz. Try any order of polynomial basis as you want and test your models on a validation set. For simplicity, please use a subset of the training set for validation to finalize your model (no cross-validation). Hint: you could increase the order until observing overfitting or performance stagnation.

Answer 5.2: Using the 2-D whitened PCA features, a polynomial regression model was trained to predict the year of made. Different polynomial degrees (1 – 6) were tested, and validation MSE was measured on a held-out subset (80/20 split).

Polynomial Degree	**Validation MSE**
1	11326.51
2	10068.77
3	8473.41
4	8612.99
5	**6968.83**
6	12430.31

Table 1: Validation MSE across polynomial degrees. The best model occurs at degree 5.

The validation error decreased steadily up to degree 5, then increased at degree 6 due to overfitting. Therefore, a 5th-order polynomial was selected as the final model. The trained model and polynomial feature map were saved as `year_model.pkl` and `year_poly.pkl` respectively.

Listing 3: year_train.py – Polynomial Regression Training

```
import numpy as np
from pathlib import Path
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import joblib

# === Paths ===
data_root = Path(r"C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2
   \HW2_data\hw2ans5")
out_dir   = data_root
out_dir.mkdir(exist_ok=True)

# === Load PCA-whitened training data ===
data = np.load(data_root / "train_pca_whitened.npz")
X, y = data["x"], data["y"]

# === Train/validation split (80/20) ===
```

```
n = len(X)
idx = np.arange(n)
np.random.seed(42)
np.random.shuffle(idx)
split = int(0.8 * n)
X_train, X_val = X[idx[:split]], X[idx[split:]]
y_train, y_val = y[idx[:split]], y[idx[split:]]

# === Try polynomial degrees 1  6   ===
best_mse, best_deg, best_model = np.inf, None, None
for deg in range(1, 7):
    poly = PolynomialFeatures(degree=deg, include_bias=True)
    Xtr = poly.fit_transform(X_train)
    Xva = poly.transform(X_val)
    model = LinearRegression().fit(Xtr, y_train)
    val_mse = mean_squared_error(y_val, model.predict(Xva))
    print(f"Degree {deg}: Validation MSE = {val_mse:.3f}")
    if val_mse < best_mse:
        best_mse, best_deg, best_model, best_poly = val_mse, deg,
           model, poly

# === Save model and polynomial ===
joblib.dump(best_model, out_dir / "year_model.pkl")
joblib.dump(best_poly,  out_dir / "year_poly.pkl")

with open(out_dir / "train_summary.txt", "w") as f:
    f.write(f"Best polynomial degree: {best_deg}\n")
    f.write(f"Validation MSE: {best_mse:.4f}\n")

print(f"    Training complete     Best degree = {best_deg},
   Validation MSE = {best_mse:.4f}")
```

5.3 Test. Write the code "year_test.py" to evaluate your model on vgg16_test.npz. Report a test MSE score and identify the image where your model shows the most accurate prediction and the image where it shows the least accurate prediction (report image filenames).

Answer 5.3: The final model was evaluated on unseen test data (`vgg16_test.npz`). Principal components were computed using the training PCA parameters to ensure consistent whitening. The mean-square-error (MSE) metric was used to measure performance:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Results:

- **Test MSE:** 6536.70
- **Most Accurate Prediction:** `8458_landscape-1892.jpg` (True = 1892.0, Pred = 1892.18)
- **Least Accurate Prediction:** `847_polyptych-of-st-luke-1455.jpg` (True = 1455.0, Pred = 1883.77)

The model achieves a mean-square error of approximately 6536.7. The most accurate prediction corresponds to an artwork created in 1892, where the predicted year nearly matches the true value. The least accurate case belongs to a 1455 artwork, whose stylistic characteristics resemble later Renaissance works, causing the model to overestimate its creation year.

Listing 4: year_test.py – Model Evaluation on Test Set

```
import numpy as np
import joblib
from pathlib import Path
from sklearn.decomposition import PCA
from sklearn.metrics import mean_squared_error

# === Paths ===
data_root = Path(r"C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2
   \HW2_data\P5_data")
out_dir   = Path(r"C:\Users\RAJ RUTGERS\Desktop\Machine Learning\HW2
   \HW2_data\hw2ans5")

# === Load train/test data ===
train = np.load(data_root / "vgg16_train.npz")
test  = np.load(data_root / "vgg16_test.npz", allow_pickle=True)

X_train, y_train = train["logit"], train["year"]
X_test_full, y_test, filenames = test["logit"], test["year"], test["
   filename"]

# === PCA ===
pca = PCA(n_components=2, whiten=True, random_state=42)
pca.fit(X_train)
X_test = pca.transform(X_test_full)

# === Load model and polynomial ===
model = joblib.load(out_dir / "year_model.pkl")
poly  = joblib.load(out_dir / "year_poly.pkl")

# === Predict ===
X_poly = poly.transform(X_test)
y_pred = model.predict(X_poly)
mse = mean_squared_error(y_test, y_pred)
```

```
print("    Test MSE:", round(mse, 4))

# === Best/Worst Predictions ===
errors = np.abs(y_pred - y_test)
best_idx, worst_idx = np.argmin(errors), np.argmax(errors)

best_file  = filenames[best_idx]
worst_file = filenames[worst_idx]

with open(out_dir / "test_results.txt", "w") as f:
    f.write(f"Test MSE: {mse:.4f}\n")
    f.write(f"Most accurate prediction: {best_file} (True = {y_test[
       best_idx]}, Pred = {y_pred[best_idx]:.2f})\n")
    f.write(f"Least accurate prediction: {worst_file} (True = {
       y_test[worst_idx]}, Pred = {y_pred[worst_idx]:.2f})\n")

print("Results saved to:", out_dir / "test_results.txt")
```